

Sistema de Gestão de Biblioteca: BiblioTech

1. Documento de Visão do Sistema

Objetivo do Sistema: O BiblioTech tem como finalidade automatizar e otimizar todo o gerenciamento do acervo físico de uma biblioteca. O sistema controla de maneira integrada o cadastro de livros, categorias literárias, autores, editoras e usuários, além de monitorar o ciclo de vida completo dos empréstimos. A solução visa garantir a consistência dos dados, impedir a retirada de livros indisponíveis e fornecer um histórico confiável de movimentações.

Descrição do Problema: Atualmente, o controle manual ou descentralizado do acervo gera desorganização crônica, falta de controle sobre prazos de devolução, perda física de exemplares e dificuldade de consulta por parte dos leitores. Adicionalmente, a ausência de uma validação automática permite que livros já emprestados sejam alocados incorretamente, inviabilizando uma gestão eficiente do fluxo interno.

Perfis de Usuários (Atores)

- **Bibliotecário (Admin):** Usuário gestor que possui privilégios totais no sistema. É responsável por realizar o CRUD completo de todas as entidades base (Usuários, Categorias, Autores, Editoras e Livros), além de supervisionar e auditar os empréstimos registrados.
- **Leitor (Cliente):** Usuário final da biblioteca. Possui acesso restrito para consultar o catálogo do acervo por meio de pesquisas parciais por título e realizar solicitações de empréstimo de obras que estejam efetivamente disponíveis.

2. Lista de Operações (Requisitos Funcionais)

Conforme as regras do projeto e a arquitetura implementada, as seguintes operações de negócio estão mapeadas e disponíveis no escopo do sistema:

1. **Autenticação de Usuários:** Controle de acesso permitindo entrar (Login) e sair (Logout) do sistema com base nas credenciais (e-mail e senha) e verificação do perfil (admin ou leitor).
2. **Gerenciamento (CRUD) de Entidades:** Operações completas de Inserir, Listar, Atualizar e Excluir para as seis entidades do modelo: **Usuario, Categoria, Autor, Editora, Livro e Empréstimo**.
3. **Operação de Associação 1:N:** No ato do cadastro de um **Livro**, o sistema realiza a vinculação obrigatória com IDs existentes das classes **Categoria, Autor e Editora**. Do mesmo modo, a classe **Empréstimo** associa-se diretamente a um **Livro** e a um **Usuario**.
4. **Operação de Pesquisa Parcial:** Mecanismo que permite listar parcialmente os livros armazenados no acervo através de um termo ou palavra informada pelo usuário (busca realizada de forma insensível a maiúsculas/minúsculas no atributo título).

5. **Regra de Negócio Complexa Multi-Entidade:** Lógica que manipula duas ou mais classes de persistência em um único fluxo de execução, garantindo a integridade transacional do negócio (detalhada na seção abaixo).

Regra de Negócio Crítica: Realização de Empréstimo

Ao acionar o método de registro de empréstimo, o sistema abre o arquivo de persistência de livros e verifica se a obra correspondente ao ID informado existe e se o atributo `disponivel` é verdadeiro (`True`). Caso positivo, o sistema altera o status do livro para falso (`False`), salvando a atualização no arquivo `livros.json`, e consolida o novo registro no arquivo `emprestimos.json`. Se o livro estiver indisponível, a transação é cancelada e o empréstimo é rejeitado.

3. Especificação Teórica para o Diagrama de Casos de Uso

Para construir visualmente o Diagrama de Casos de Uso (exigência de 5 pontos da Tarefa 1), utilize os elementos mapeados na tabela abaixo, estabelecendo as conexões entre os Atores e os Casos de Uso (elipses):

ATOR	CASOS DE USO ASSOCIADOS (ELIPSES)	RELACIONAMENTO / REGRAS
Bibliotecário (Admin)	• CRUD de Usuários • CRUD de Categorias • CRUD de Autores • CRUD de Editoras • CRUD de Livros • Listar/Excluir Empréstimos	Acesso completo e irrestrito de escrita e leitura de dados.
Leitor (Cliente)	• Pesquisar Livro (Nome Parcial) • Solicitar Empréstimo de Livro • Visualizar Meu Histórico de Empréstimos	Acesso restrito a consultas e operações próprias.
Gerais (Ambos os Atores)	• Efetuar Login • Efetuar Logout	Os casos de uso de operações incluem a relação de dependência <<include>> apontando para "Efetuar Login".

4. Especificação do Diagrama de Classes (Modelo e Persistência)

Esta seção descreve exatamente a assinatura estrutural das classes Python implementadas. As tabelas especificam os atributos e os métodos que devem constar nas caixas do seu Diagrama de Classes UML (exigência de 10 pontos da Tarefa 1).

4.1. Classes do Modelo (Módulo 'Modules')

NOME DA CLASSE	ATRIBUTOS (MEMBROS)	MÉTODOS IMPLEMENTADOS
Autor	+ id: int + nome: str + nacionalidade: str	+ __init__(id_autor, nome, nacionalidade) + to_dict(): dict + from_dict(dados: dict): Autor
Categoria	+ id: int + nome: str	+ __init__(id_categoria, nome) + to_dict(): dict + from_dict(dados: dict): Categoria
Editora	+ id: int + nome: str + cidade: str	+ __init__(id_editora, nome, cidade) + to_dict(): dict + from_dict(dados: dict): Editora
Usuario	+ id: int + nome: str + email: str + senha: str + perfil: str ('admin' 'leitor')	+ __init__(id_usuario, nome, email, senha, perfil) + to_dict(): dict + from_dict(dados: dict): Usuario
Livro	+ id: int + titulo: str + id_categoria: int (Associação) + id_autor: int (Associação) + id_editora: int (Associação) + disponivel: bool = True	+ __init__(id_livro, titulo, id_categoria, id_autor, id_editora, disponivel) + to_dict(): dict + from_dict(dados: dict): Livro
Emprestimo	+ id: int + id_livro: int (Associação) + id_usuario: int (Associação) + data_retirada: str + ativo: bool = True	+ __init__(id_emprestimo, id_livro, id_usuario, data_retirada, ativo) + to_dict(): dict + from_dict(dados: dict): Emprestimo

4.2. Classes de Persistência (Módulo 'DAO')

A persistência baseia-se em uma classe abstrata mãe contendo os métodos genéricos de manipulação de arquivos JSON, herdada pelas subclasses específicas de cada entidade do modelo.

NOME DA CLASSE	ATRIBUTOS DE CLASSE (ESTÁTICOS)	MÉTODOS DE CLASSE (MAPEAMENTO CRUD)
DAO (Classe Super/Mãe)	+ arquivo: str = "" + objetos: list = [] + classe_modelo: Class = None	+ inserir(obj): void + listar(): list + atualizar(obj_atualizado): void + excluir(id_obj): void + salvar(): void + abrir(): void
AutorDAO	+ arquivo: str = "autores.json" + objetos: list = [] + classe_modelo = Autor	Herança total de DAO
CategoriaDAO	+ arquivo: str = "categorias.json" + objetos: list = [] + classe_modelo = Categoria	Herança total de DAO
EditoresDAO	+ arquivo: str = "editoras.json" + objetos: list = [] + classe_modelo = Editora	Herança total de DAO
UsuarioDAO	+ arquivo: str = "usuarios.json" + objetos: list = [] + classe_modelo = Usuario	Herança total de DAO
LivroDAO	+ arquivo: str = "livros.json" + objetos: list = [] + classe_modelo = Livro	+ pesquisar_por_titulo(termo: str): list (Método específico para pesquisa parcial)
EmprestimoDAO	+ arquivo: str = "emprestimos.json" + objetos: list = [] + classe_modelo = Emprestimo	+ registrar_emprestimo(novo_emprestimo): bool (Método específico da regra multientidade)

5. Estrutura de Arquivos e Código de Teste (20 Pontos)

Para obter os 20 pontos atribuídos à codificação, o projeto deve ser entregue estruturado em pacotes isolados, mantendo a conformidade com o script de testes automatizado fornecido (teste.py), o qual valida a gravação, persistência transacional de arquivos e leitura correta de objetos dinâmicos.

Organização do Diretório Principal:

- /Modules: Contém subpastas para cada entidade do domínio, abrigando as classes de modelo (ex: Autor.py, Livro.py) e as classes de persistência filhas (ex: AutorDAO.py, LivroDAO.py).
- /DAO: Contém o arquivo genérico DAO.py com a lógica central de serialização/deserialização em JSON.

- `/Jsons`: Diretório físico onde os arquivos persistidos (`livros.json`, `usuarios.json`, etc.) são manipulados em tempo de execução de forma isolada.
- `teste.py`: Arquivo de script na raiz que executa sequencialmente o fluxo de testes exigido pelo professor.